MagicPiG: sParse Inference enGine for LLM

July 26, 2024

Abstract

1 Introduction

Large language models (LLMs) (e.g. GPT [2], Gemini [13], Llama [14]) and their downstream applications continue to gain attention. These applications are expensive due to the massive resources required for the giant models [6]. A well-known way to alleviate this situation is batching multiple requests to amortize the parameter accessing cost [6, 18]. However, LLMs store previous key-value states (KV cache) to avoid re-computation while decoding. The increasing model parameters and context sizes, which lead to the growing in KV cache memory consumption, prohibit the usage of large batch size in LLM serving. For example, an H100-80GB GPU can only host a batch of size 4 when serving a Llama3-70B [1] model with 8k context size with INT8 quantization.

Recent works [4, 7, 12, 16, 19] have been proposed to tackle the problem of KV cache memory consumption. However, they still suffer from three major limitations. (1) **Accuracy dropping.** Compression methods like H2O [19], StreamingLLM [16] and SnapKV [7] perform an irreversible eviction on KV cache. Since the eliminated keys and values can never come back for future token generation, information loss is inevitable and can result in significant accuracy dropping in certain tasks. (2) **Out of Memory.** Another track of works such as Loki [10] and Quest [12] get rid of eviction by saving all the KV cache but performing a dynamic sparse attention. Although this kind of methods potentially solve the problems of information loss, the allowed batch size is limited, which also prevents their applications when VRAM is a scarce resource. (3) **High Cost.** FastDecode [4] saves KV cache in DRAM. To mitigate the bandwidth gap between CPUs and GPUs, external CPU resources are required, which is usually not achievable in GPU servers and will increase cost for extra CPUs. Moreover, GPUs with HBM (e.g. A100) can widen this bandwidth gap, which is intractable by CPUs without algorithm changing.

An ideal lost-cost serving system should (1) maintain high accuracy across various tasks; (2) be memory-friendly to enable high parallelism; (3) not depend on external resources. These goals can be easily achieved by a GPU with infinite VRAM, which is impractical with current hardware. However, the memory capacity of CPUs, which can be considered as almost infinite compared with the memory footprint introduced by LLM, is underutilized. Two reasons obstruct leveraging CPUs in LLM serving. (1) **Low computation capacity.** Compared to modern GPUs with thousands of tensor cores, the computation capacity of CPUs is usually constraint and typically only $1\% \sim 2\%$ [4] of GPUs, making it extremely slow for arithmetic-intensive operators. (2) **Slow communication with GPUs.** Currently, most clusters connect CPUs and GPUs with PCIE4$\times$16, which can only provide 31.5GB/s bandwidth, preventing large amount of data transmission between CPUs and GPUs.

The dilemma of memory and computation, as well as the slow communication requires a rethinking the role of CPUs in LLM serving. Specifically,

- *CPUs have higher memory bandwidth.* Compared to its limited computation, CPU usually has a relatively higher memory bandwidth, typically 80GB/s $\sim$ 200GB/s, which is $2.5 \sim 6\times$ of PCIEs bandwidth and roughly $10\% \sim 20\%$ of VRAM bandwidth in GPU. This feature makes CPUs more suitable for bandwidth bound workload, which is also pointed out in [4].

- *Attention is always bandwidth-bound.* The arithmetic intensity of linear projections is increasing with batch size. However, that of attention, (i.e. $h = \text{softmax}(qk^T)v$) is invariant of batch size. For Multi-Head-Attention (MHA) models (e.g. Llama2-7b,13b [14]), the arithmetic intensity is 1 and for Group-Query-Attention (GQA) models (e.g. Llama3-8B,70B [1]), is typically $4 \sim 8$. Considered the FLOPs and bandwidth of modern hardware (especially GPUs), attention computation is always bandwidth bound.

- *Attention is sparse.* As studied by previous works, only a small proportion of KV cache can approximate attention precisely at most of the time. Dynamic sparse attention can further boost accuracy [7, 10, 12, 19].

With the above insights, a heterogeneous system with sparse attention on CPUs seems to meet the requirements of low cost LLM serving. However, several technical challenges require addressed. First, to reduce workload while maintaining accuracy, we must smartly retrieve the most related KV cache and accurately estimate the distribution of attention scores. Second, retrieval process should be fast to reduce overhead. Third, we need to properly manage the fast but scarce VRAM resources to fully leverage GPUs.

This paper presents MagicPiG, a heterogeneous LLM serving system with dynamic sparsity. Three key innovations are introduced to overcome the aforementioned challenges.

- **LSH for KV Retrieval.** MagicPiG leverages locality sensitive hashing (LSH) for KV cache retrieval. Benefiting from (1) matrix-multiplication based hashing encoding on GPUs and (2) multi-threading searching on CPUs, LSH can efficiently and accurately retrieve most related KV cache.

- **Sampling based Attention Approximation.** Thanks to the mathematical features of LSH, the probability of each key-value pair being retrieved can be accurately calculated, which enables MagicPiG to better approximate the attention output beyond the normal top-k based sparse attention, thus maintaining a high accuracy for various tasks.

- **Hierarchical KV Cache Management.** Inspired by previous works on KV Cache compression [9, 15], the most important keys and values are selected as *persistent memory* to store on device while others on DRAM as *volatile memory*. Dynamic sparse attention is performed at inference time. Through hierarchical KV cache management, MagicPiG significantly reduces the workload on CPUs.

2 Background

2.1 LLM inference

Transformer models are a stack of Transformer layers, which have two major blocks.

MLP block is consist of W_{up}, W_{gate} and W_{down} and can be formulated as:

$$y = (\mathcal{ACT}(xW_{gate}) * (xW_{up}))W_{down} \tag{1}$$

$\mathcal{ACT}$ is an activation function which is SiLU for Llama [1, 14] models and $*$ denotes element-wise multiplication.

Attention block applies linear projection on input hidden states x_i.

$$q_i = x_i W_Q, \quad k_i = x_i W_k, \quad v_i = x_i W_v \tag{2}$$

k_i,v_i of new input hidden states will be concatenated with previous calculated key-value pairs (i.e. KV cache) by

$$K = concat(K, k_i) \quad V = concat(V, v_i) \tag{3}$$

The attention scores are computed by the inner product of q_i and all previous key vectors stored in K.

$$A_i = \text{Softmax}(q_i K^T / \sqrt{D}) \tag{4}$$

where D is head dimension. After obtaining the attention score, a weighted-average is performed among value vectors stored in V.

$$u_i = A_i V \tag{5}$$

Finally, an output projection is applied.

$$o_i = u_i W_o \tag{6}$$

.

2.2 Locality sensitive hashing

Locality sensitive hashing (LSH) is a family of hashing functions which can make similar inputs collide with higher probability than non-similar ones. Theoretically, consider $\mathcal{H}$ to be a family of hashing functions mapping R^D to S.

Definition 2.1 (LSH family)**.** A family $\mathcal{H}$ is called (S_0, cS_0, p_1, p_2)-sensitive if for any two points $x, y \in R^D$, and h chosen uniformly from $\mathcal{H}$ satisfies the following: (1) if $Sim(x,y) \geq S_0$ then $Pr(h(x) = h(y)) \geq p_1$; (2) if $Sim(x,y) \leq cS_0$ then $Pr(h(x) = h(y)) \leq p_2$.

One sufficient condition for a hash family $\mathcal{H}$ to be an LSH family is that the collision probability $Pr_{\mathcal{H}}(h(x)=h(y))$ should be a monotonically increasing with the similarity, i.e.

$$Pr_{\mathcal{H}}(h(x)=h(y))=f(Sim(x,y)) \tag{7}$$

where f is a monotonically increasing function. In fact, most of the popular known LSH families, such as Simhash and WTA hash, satisfy this strong property.

LSH Search. The LSH algorithm uses two parameters, (K,L). We construct L independent hash tables. Each hash table has a meta-hash function H that is formed by concatenating K random independent hash functions from the collection $\mathcal{F}$. Given a query, we collect one bucket from each hash table and return the union of L buckets. Intuitively, the meta-hash function makes the buckets sparse and reduces the number of false positives, because only valid nearest-neighbor items are likely to match all K hash values for a given query. The union of the L buckets decreases the number of false negatives by increasing the number of potential buckets that could hold valid nearest-neighbor items. The candidate generation algorithm works in two phases:

- **Pre-processing Phase:** We construct L hash tables from the data by storing all elements x. We only store pointers to the vector in the hash tables because storing whole data vectors is very memory inefficient.
- **Query Phase:** Given a query Q, we search for its nearest-neighbors. We report the union from all of the buckets collected from the L hash tables. Note that we do not scan all the elements but only probe L different buckets, one bucket for each hash table.

SimHash. SimHash is the LSH family for the Cosine Similarity metric, which is the angle between two vectors $x,y \in R^D$, i.e. $\theta = \arccos \frac{xy^T}{||x||||y||}$. For a vector $x \in R^D$, SimHash generates a random hyperplane w and returns the sign of the projection of x onto w. Two vectors share the same sign only if the random projection does not fall in-between them. Since all angles are equally likely for a random projection, the probability that two vectors x, y share the same sign for a given random projection is $p = 1 - \frac{\theta}{\pi}$. For L hash tables each with K random hash functions, the probability of y to be retrieved by query x is $1-(1-p^k)^L$.

3 Case Study: Why go dynamic

In this section, we analysis one important drawback of static KV cache compression methods. Once the token has been evicted, it can never come back in the static methods. However, there exists certain situations, we cannot decide the importance of KV cache in advance. Here we take a recent state-of-the-art static KV compression method, SnapKV [7] and Llama3-70B-Instruct as an example. We use a synthetic task to demonstrate how previously unimportant tokens can influence the quality of generation.

Prompt System: You are an assistant. Your goal is to be helpful, friendly and answer the questions as more as you can. User: Hello, I want to test your mathematic ability, nice to meet you! I have a problem for you. @ $<<< assign \quad v_0 = v_2 - 1 >>>$ @@ $<<< assign \quad v_1 = v_2 + 1 >>>$ @@ $<<< assign \quad v_2 = 1 >>>$ @ **Question:** Can you tell me which variables are equal to 2 in those assignments in '<<<>>>'? The values of those variables can be used before definition as they are constant. You can solve the problem in a step-by-step way. Assistant:

Answer from SnapKV(20% Sparsity) Let's break down the assignments step by step: 1. $<<< assign >>> v <<< assign >>> = 1$ We have $v = 1$. 2. $<<< assign >>> = -1$ We have $= 0$. 3. $<<< assign >>> = 1 - 1$ We have $= 0$. Now, the question is which variables have a value of 1. The answer is v. So, the variable with a value of 1 is v.

In this case, we found the indices of varibles are discarded while KV selection. During decoding, the model cannot differentiate different varibles, making it impossible to output correct answer. As a comparison, we also show the anwer from the actual topk attention.

Answer from TopK(5% Sparsity) Let's break down the assignments step by step: 1. $v_2 = 1$ (initial value of v_2 is set to 1) 2. $v_1 = v_2 + 1$ (v_1 is assigned the value of $v_2 + 1$, which is 1 + 1 = 2) 3. $v_0 = v_2 - 1$ (v_0 is assigned the value of $v_2 - 1$, which is 1 - 1 = 0) So, after these assignments, the values of the variables are: - $v_0 = 0, v_1 = 2, v_2 = 1$ Therefore, the variable that is equal to 2 is v_1.

4 Attention Approximation

4.1 Problem formulation

Given $q \in R^{1\times D}$, $K=[k_1,k_2,...,k_n] \in R^{n\times D}$ and $v=[v_1,v_2,...,v_n] \in R^{n\times D}$, self-attention procedure can be formulated as

$$h = \mathrm{Attn}(q,K,V) = \mathrm{softmax}(qK^T)V \tag{8}$$

Softmax scale $\frac{1}{\sqrt{D}}$ is ignored for simplicity. In order to reduce workload (especially memory access), we need to constraint our computation to S, which is a subset of $[n]$ and reduce the approximation error in the meantime by solving

$$\begin{aligned} &\underset{\mathrm{F},S}{\mathrm{argmin}}||h-\widetilde{h}|| \quad \text{s.t.} \\ &\widetilde{h} = \mathrm{F}(q,K_S,V_S), \quad S \subseteq [n], \quad |S| = \mathcal{K} \end{aligned} \tag{9}$$

where F denotes an approximation function and $\mathcal{K}>0$ is computation budget.

4.2 LSH for KV retrieval

Intuitively, given the budget $|S|=\mathcal{K}$, we want to retrieve the elements with the highest attention scores, i.e, $S := \mathrm{Top}\mathcal{K}_{i\in[n]}(qk_i^T)$, which is equivalent to maximum inner product search (MIPS) between q and $k_1,k_2,...k_n$.

To solve the MIPS, we perform the following transformation

$$\begin{aligned} \widetilde{k_i} &= k_i - \frac{\sum_{i=1}^n k_i}{n} \\ C &= \max_{i\in[n]}||\widetilde{k_i}|| \\ \bar{k_i} &= [\widetilde{k_i}, \sqrt{C^2 - ||\widetilde{k_i}||^2}] \\ \bar{q} &= [q,0] \end{aligned} \tag{10}$$

Notice that

$$qk_i^T = q\widetilde{k_i}^T + M = \bar{q}\bar{k_i}^T + M = C||q||\cos(\bar{q},\bar{k_i}) + M \tag{11}$$

Where $M = q(\frac{\sum_{i=1}^n k_i}{n})^T$ and is a constant across k_is. Then the maximum inner production between q and k_is is transformed to a nearest neighbour search problem among $\bar{q}$ and $\bar{k_i}$s, with Cosine Similarity metric, which can be effectively solved by **SimHash**.

Empirical Verification. To demonstrate the effectiveness of LSH based KV retrieval algorithm, we collect 128 examples from 2wikimqa_e, each with 2048 prefilled tokens. We evaluate recall@1(Figure 1a),recall@10(Figure 1b)and **Recovery Rate** ($\mathcal{R}$)(Figure 1c) by our retrieval method, which is defined by

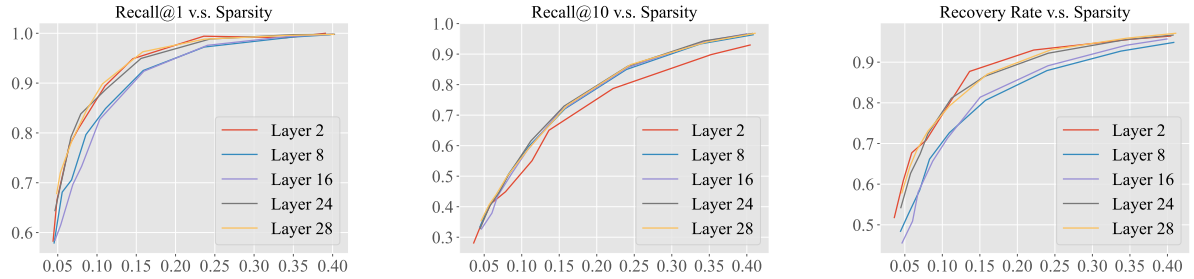
$$\mathcal{R} = \frac{\sum_{i\in S} e^{q_i k_i^T/\sqrt{D}}}{\sum_{i\in[n]} e^{q_i k_i^T \sqrt{D}}} \tag{12}$$

where S denotes the retrieved subset and D is the head dimension. $\mathcal{R}$ can be interpreted as the proportion of attention score occupied by retrieved subset after softmax. Sparsity is controlled by increasing L while fix K as 11. **Takeaway:** With only 10% memory access, we have $\geq$80% probability to obtain the most important key-value pair and recover $\geq$70% attention scores.

4.3 Sampling Based Estimation

In addition to selecting proper S, F also needs to be determined for precise approximation. A straightforward practice is directly using self-attention procedure to the retrieved subset by

$$\widetilde{h} = \mathrm{F}(q,K_S,V_S) = \mathrm{softmax}(qK_S^T)V_S \tag{13}$$



(a) Recall@1 v.s. Sparsity (b) Recall@10 v.s. Sparsity (c) Recovery Rate v.s. Sparsity

Figure 1: Recall@1(**Left**), recall@10(**Mid**) and Recovery Rate(**Right**) are evaluated for our LSH based KV retrieval method. With only 10% memory access, we have $\geq 80\%$ probability to obtain the most important key-value pair and recover $\geq 70\%$ attention scores. The traces are collected by 128 examples from 2wikimaq_e, with Llama-2-7b-chat model. Sparsity is controlled by increasing L while fix K as 11.

However, by SimHash, we can obtain the probability of certain element $i \in [n]$ being retrieved (noted by p_i), thus improving the estimation of h. By unrolling Equation (8), we have

$$h = \frac{1}{Z} \sum_{i=1}^{n} e^{qk_i^T} v_i$$

$$Z = \sum_{i=1}^{n} e^{qk_i^T} \tag{14}$$

Z is also known as **paritition function**, which can be effectively estimated with K_S,V_S and corresponding probability p_i,$i \in S$ [11] by

$$\widetilde{Z} = \sum_{i \in S} \frac{e^{qk_i^T}}{p_i} \tag{15}$$

We can further prove the estimation is unbiased as

$$\mathbb{E}[\widetilde{Z}] = \mathbb{E}[\sum_{i \in S} \frac{e^{qk_i^T}}{p_i}] = \mathbb{E}[\sum_{i \in [n]} \mathbb{1}(i \in S) \frac{e^{qk_i^T}}{p_i}] = \sum_{i \in [n]} \frac{e^{qk_i^T}}{p_i} \mathbb{E}[\mathbb{1}(i \in S)] = \sum_{i \in [n]} e^{qk_i^T} = Z \tag{16}$$

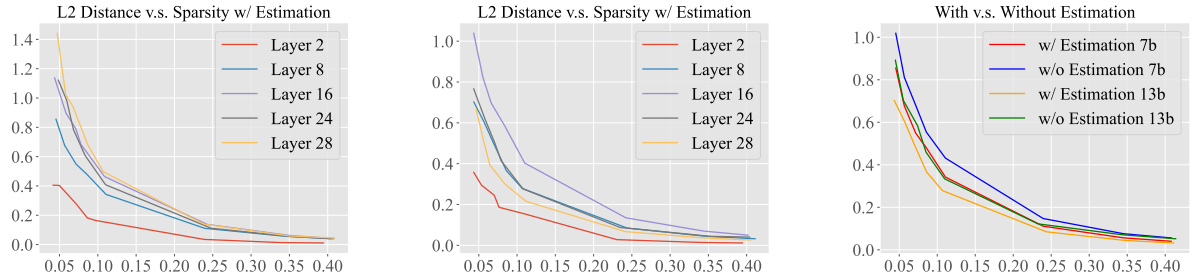
Similarly, h can be estimated by

$$\widetilde{h} = \frac{1}{\widetilde{Z}} \sum_{i \in S} \frac{e^{qk_i^T}}{p_i} v_i = \text{softmax}_{i \in S}([qk_i^T - \log p_i]) V_S \tag{17}$$

Empirical Verification. To demonstrate the effectiveness of LSH based KV retrieval algorithm, we collect 128 examples from 2wikimqa_e, each with 2048 prefilled tokens. We evaluate the numerical error of the approximated attention output on each head, i.e. the L_2 distance from the ground truth $||\widetilde{h}_j - h_j||$, where j denotes the head id. Sparsity is controlled by increasing L while fix K as 11. **Takeaway:** (1) The numerical error is almost converged with 25% memory access to previous key-value pairs. (2) LSH based estimation reduces numerical error by up to 27% and constantly improve the approximation accuracy across sparsity and models.

4.4 Leveraging features of LLMs

As a feature of LLMs, prior works point out a static subset of prefilling KV cache usually concentrates a large propotion of attention scores during generation and is sufficient to achieve high accuracy in many tasks. To leverage this feature, we also fix a static subset U (disjoint with S) for attention computation every time. And



(a) L_2 distance v.s. Sparsity (7b) (b) L_2 distance v.s. Sparsity (13b) (c) Effectiveness of Estimation

Figure 2: We show the numerical error (i.e. the L_2 distance from the ground truth attention output) of 7b (**Left**) and 13b (**Mid**) models. The numerical error is almost converged with 25% memory access to previous key-value pairs. We also compare the numerical errors between with and without LSH based estimation in the 8th layer (**Right**). Estimation algorithm reduces numerical error by up to 27% and constantly improve the approximation accuracy across sparsity and models. The traces are collected by 128 examples from 2wikimaq_e, with Llama-2-7b/13b-chat models. Sparsity is controlled by increasing L while fix K as 11.

the estimation of h can be modified with

$$\widetilde{h} = \frac{1}{\widetilde{Z}} \sum_{i \in S \cup U} \frac{e^{qk_i^T}}{p_i} v_i = \mathrm{softmax}_{i \in S \cup U}([qk_i^T - \log p_i]) V_{S \cup U}$$

$$\widetilde{Z} = \sum_{i \in S \cup U} \frac{e^{qk_i^T}}{p_i} \tag{18}$$

As U is determined during prefilling (deterministic at decoding phase), we have $p_i = 1, \forall i \in U$.

Empirical Verification. To demonstrate the effectiveness of LSH based KV retrieval algorithm, we collect 128 examples from 2wikimqa_e, each with 2048 prefilled tokens. We evaluate the numerical error of the approximated attention output. In the experiments, SnapKV + LSH leverages 10% static tokens obtained by SnapKV while prefilling and other tokens are retrieved by LSH. **Takeaway:** (1) With small computation budget, static methods based on heuristics on prefilling attention scores (H_2O and SnapKV [7, 19]) or token positions (StreamingLLM [16]) are more accurate. (2) Incorporating static KV cache significantly improve the accuracy of attention approximation over LSH based retrieval. (3) With LSH retrieved KV cache, the approximation error decreases faster than static methods when increasing the computation budget, showing a better scalability.

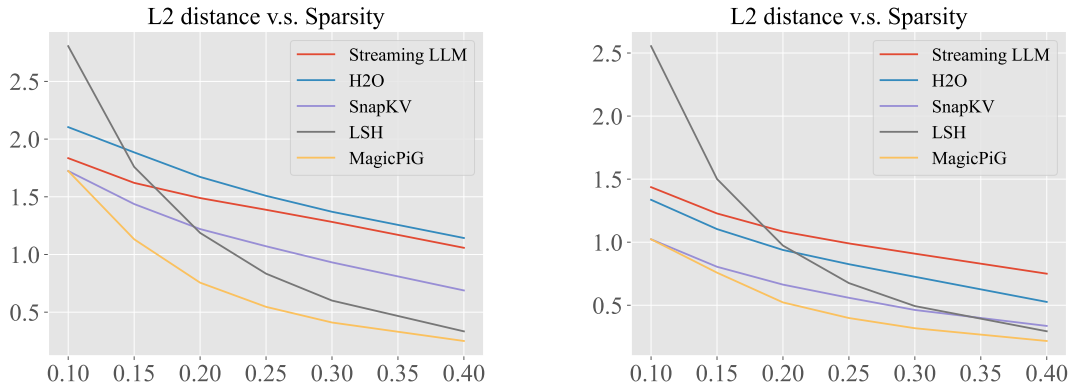
4.5 Summary

Our LSH based attention approximation algorithm can be partitioned into two phases.

- **Prefilling:** A certain proportion of KV cache is selected as static cache (or *persistent memory*) based on heuristics on prefilling attention scores. For the rest of KV cache (or *volatile memory*), we build hash tables with transformations on K as Equation (10), prepared for search at decoding time.
- **Decoding:** q is encoded for LSH retrieval. With persistent memory and retrieved KV cache, attention output is estimated with Equation (18).

5 MagicPiG

In this section, we present the system design of MagicPiG (Section 5.1), which mainly consists of a hybrid attention server(Section 5.3) and a hybrid LSH retriever(Section 5.2).



(a) L_2 distance v.s. Sparsity (7b)

(b) L_2 distance v.s. Sparsity (13b)

Figure 3: We show the numerical error (i.e. the L_2 distance from the ground truth attention output) of 7b (**Left**) and 13b (**Mid**) models in the 8th layer. With LSH retrieved KV cache, the approximation error decreases faster than static methods when increasing the computation budget, showing a better scalability.

5.1 System Overview

MagicPiG's system overview is shown in Figure 4. Similar to [4], we also partition the workload into parametric part (including MLP, QKV projections, O projections, layer norms and input/output embedding) and non-parametric part ($h = \text{softmax}(qK^T)V$). The former is totally running on GPU while the latter is executed in a hybrid way with MagicPiG.

Memory Layout. The static KV cache (persistent memory), model parameters and hashing functions are stored on GPU while the other dynamic activated KV cache (volatile memory) is stored on CPU, together with corresponding hash tables.

Execution Steps. Once q is generated by the model, MagicPiG will execute the following steps to get the attention output back to the model. **1. Hashing Encoding.** The hash code of q is calculated via hashing functions stored on GPU. **2. Retrieval.** With the hash code of q sent to CPU, LSH retriever search with the hash tables, which are already built during prefilling phase. **3. CPU Attention Approximation.** with the retrieved indices, attention server estimate the partial attention output via Equation (18). **4. GPU Attention Calculation.** latest generated k,v are saved in GPU KV slots and another partial attention is performed with static cache on GPU; **5. Attention Merging.** Two partial attention outputs are merged and return back to models as o.

5.2 LSH Retriever

In this section, we introduce the challenges in LSH search deployment in KV retrieval and discuss how MagicPiG overcome the challenges through algorithm-system co-design.

Accuracy-Efficiency Tradeoff. The search accuracy of LSH relies on the hyper-parameters, i.e. (K,L). Typically, larger (K,L) will lead to a more accurate search result. However, (1) a large amount of extra FLOPS requires to be spent on hashing encoding when (K,L) is large; (2) large (K,L) will introduce a high memory occupation to store hash tables; (3) The retrieval process is not vectorized, (e.g. different number of tokens are retrieved in different heads) which is difficult for GPU.

Solutions. To best leverage the computation of GPU and memory capacity of CPU, we make the following design. (1) We place the hashing functions (random projection matrices in SimHash) on GPUs, thus we can accelerate the hashing encoding process. As random hashing matrices can be shared across layers and requests, GPU memory occupation only increases by $K \times L \times D$ (which is usually smaller than one Q projection module) where D is the model dimension. (2) Hash tables are stored on CPU and search process is also conducted on CPU. With $L > 128$, hash tables and related memory for fast search will be larger than KV cache, making

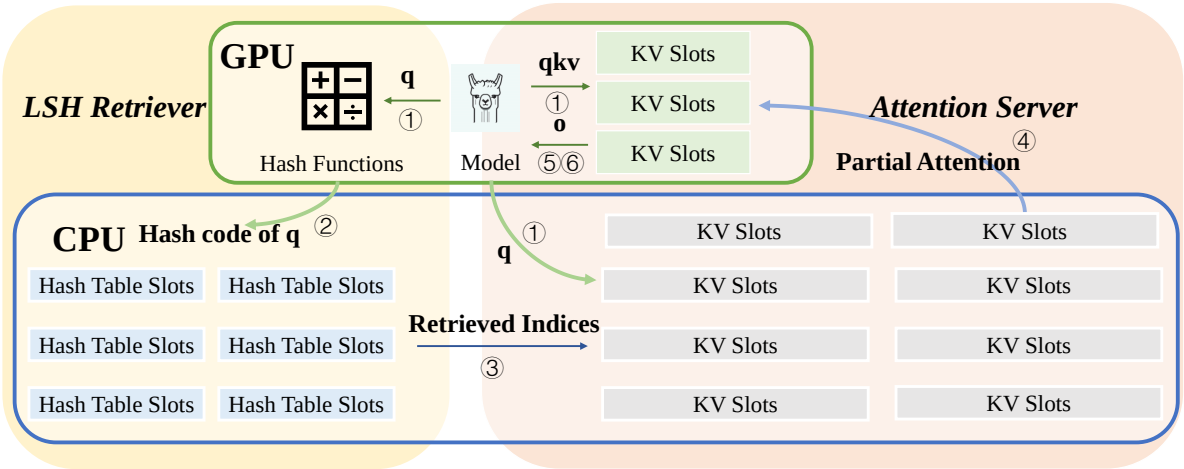


Figure 4: **System Overview**. MagicPiG consists of two major parts, **LSH Retriever** and **Attention Server**. The static KV cache (persistent memory), model parameters and hashing matrices are stored on GPU while the other dynamic activated KV cache (volatile memory) is stored on CPU, together with corresponding hash tables. During decoding, (1) q,k,v is generated by model on GPU and sent to LSH retriever and attention server; (2) the hash code of q is generated by hash functions on GPU and sent to CPU for retrieval (3) attention server gets the retrieved KV cache indices and performs partial attention on CPU; (4) partial attention results are sent to GPU; (5) latest generated k,v are saved in GPU KV slots and another partial attention is performed with static cache on GPU; (6) Two parts of attention output is combined and return back to the model as o. GPU and CPU communicate by PCIE.

CPU search a must choice. Moreover, compared to GPU's proficiency in matrix multiplication operators, multi-threading of CPU can be fully leveraged for fast unstructured search.

5.3 Attention Server

Our attention server has two parts. GPU part manages the persistent memory and CPU part manages the volatile memory. Considering the amount of memory, transferring KV cache via slow PCIE is not tolerable. For both parts, attention is performed locally and we leverage the technique of partial attention (recursive attention [17]) to merge the final attention output. Specifically, on GPU side,

$$\begin{aligned} \widetilde{h_U} &= \mathrm{softmax}_{i\in U}([qk_i^T/\sqrt{D}])V_U \\ LSE_U &= \log(\sum_{i\in U} e^{qk_i^T/\sqrt{D}}) \end{aligned} \tag{19}$$

on CPU side,

$$\begin{aligned} \widetilde{h_S} &= \mathrm{softmax}_{i\in S}([qk_i^T/\sqrt{D} - \log p_i])V_S \\ LSE_S &= \log(\sum_{i\in S} e^{qk_i^T/\sqrt{D} - \log p_i}) \end{aligned} \tag{20}$$

After transferring $\widetilde{h_S}$ and LSE_S to GPU, attention output is merged by

$$\widetilde{h} = \mathrm{softmax}([LSE_U, LSE_S])[\widetilde{h_U}, \widetilde{h_S}] \tag{21}$$

where U is the static KV cache set on GPU and S is the retrieved KV cache set on CPU.

5.4 Implementation

MagicPiG is written in Pytorch [8] for GPU operators and data transferring between GPU and CPU. LSH retrieval and sparse attention on CPU are written in C++ and leverage BFloat16 in Fbgemm [5] and AVX512 for further acceleration.

6 Evaluations

6.1 Accuracy

6.1.1 LM EVAL HARNESS

We evaluate our methods on four generation tasks from lm-eval-harness [3]: GSM8K, GSM8K-COT and COQA to show MagicPiG maintains model's reasoning and conversational abilities in Table 1.

Conclusions. (1) MagicPiG maintains original model's abilities with a maximum of 2.8% performance dropping. (2) MagicPiG outperforms static KV selection method (SnapKV) with more budget except 2 datapoints. (3) Although SnapKV can obtain reasonable results on flexible match, it gets extremely low score on strict match, which indicates it cannot follow the required answer format.

Table 1: MagicPiG is evaluated on three generation tasks from lm-eval-harness [3]: GSM8K, GSM8K-COT and COQA. Budget has two parts, static KV cache by SnapKV (S) and dynamic KV cache retrieved by LSH (D). We use hyper-parameter (K, L) to control the dynamic sparsity.

Model	Budget (S + D)	(K,L)	**GSM8K**	**GSM8K-COT**	**COQA**
Llama-2-7b-chat	10%+6%	(11, 120)	22.4/23.1	16.7/22.4	58.2/75.4
Llama-2-7b-chat	25%+0%	N/A	5.3/20.5	1.1/19.9	56.7/74.1
Llama-2-7b-chat	100%+0%	N/A	23.7/24.0	16.2/21.5	59.9/75.9
Llama-2-13b-chat	10%+6%	(11, 120)	32.8/35.6	33.1/36.4	63.9/79.0
Llama-2-13b-chat	25%+0%	N/A	7.9/33.4	0.8/34.3	63.1/78.3
Llama-2-13b-chat	100%+0%	N/A	35.6/36.5	34.5/37.1	64.3/79.0
Llama-3-8b-Instruct	10%+10%	(11, 55)	73.7/74.4	75.7/76.4	61.0/77.8
Llama-3-8b-Instruct	25%+0%	N/A	74.2/75.2	11.4/68.1	59.4/77.4
Llama-3-8b-Instruct	100%+0%	N/A	75.4/75.4	76.9/77.5	61.8/78.6

Table 2: Throughput Improvement by MagicPiG. We measure the throughput of three different models on L40 GPU and Intel 6338 CPU. For each model, the first line is the inference speed of MagicPiG, under 10% static budget and 6% dynamic budget with $(K,L)=(11,120)$, With this hyper-parameter, MagicPiG demonstrate high accuracy in evaluation tasks.

Model	**Hardware**	Budget (S + D)	(K,L)	**LEN**	**BSZ**	**TPS**
Llama-2-7b-chat	L40+Intel 6338	10%+6%	(11, 120)	3600	64	380 (×1.62)
Llama-2-7b-chat	L40+Intel 6338	100%+0%	N/A	3600	16	235
Llama-2-7b-chat	L40 with Infini VRAM	100%+0%	N/A	3600	64/INF	395/444
Llama-2-13b-chat	L40+Intel 6338	10%+6%	(11, 120)	3600	24	163 (×1.87)
Llama-2-13b-chat	L40+Intel 6338	100%+0%	N/A	3600	6	87
Llama-2-13b-chat	L40 with Infini VRAM	100%+0%	N/A	3600	24/INF	203/284
CodeLlama-7b	L40+Intel 6338	10%+6%	(11, 120)	7600	36	189 (×1.62)
CodeLlama-7b	L40+Intel 6338	100%+0%	N/A	7600	8	117
CodeLlama-7b	L40 with Infini VRAM	100%+0%	N/A	7600	36/INF	194/211
CodeLlama-13b	L40+Intel 6338	10%+6%	(11, 120)	7600	14	87 (×2.00)
CodeLlama-13b	L40+Intel 6338	100%+0%	N/A	7600	3	43
CodeLlama-13b	L40 with Infini VRAM	100%+0%	N/A	7600	14/INF	102/133

6.1.2 LongBench

We also evaluated MagicPiG on LongBench_E for Llama-2-7b-chat and Llama-3-8b-Instruct models Table 3. Similar to the conclusions made for lm-eval-harness, MagicPiG significantly outperforms static methods and approximate the performance of original model. The evaulation is conducted within the context of the models.

Table 3: MagicPiG is evaluated on LongBench_E. Budget has two parts, static KV cache by SnapKV (S) and dynamic KV cache retrieved by LSH (D). We use hyper-parameter (K, L) to control the dynamic sparsity.

Model	Budget (S + D)	(K,L)	**MN**	**MEN**	**GOV**	**QASPER**
Llama-2-7b-chat	10%+6%	(11, 120)	24.6	42.5	27.8	19.0
Llama-2-7b-chat	5%+6%	(12, 240)	25.9	46.3	27.8	19.4
Llama-2-7b-chat	25%+0%	N/A	23.7	40.1	24.9	14.4
Llama-2-7b-chat	100%+0%	N/A	27.2	43.7	30.7	23.6
Llama-3-8b-Instruct	10%+10%	(11, 55)	25.7/24.0	44.0/35.1	27.9/26.4	32.0/25.9
Llama-3-8b-Instruct	5%+10%	(12, 105)	25.9/23.4	43.7/33.8	27.3/26.6	31.7/26.8
Llama-3-8b-Instruct	25%+0%	N/A	24.0/23.3	37.0/31.6	25.7/25.6	21.2/22.0
Llama-3-8b-Instruct	100%+0%	N/A	27.3/24.2	44.5/38.2	31.6/30.1	35.8/30.1
Model	Budget (S + D)	(K,L)	**LCC**	**RP**	**PRE**	**TREC**
Llama-2-7b-chat	10%+6%	(11, 120)	63.1	49.2	20.0	55.0
Llama-2-7b-chat	5%+6%	(12, 240)	61.2	48.6	18.0	55.0
Llama-2-7b-chat	25%+0%	N/A	60.4	49.9	21.0	47.0
Llama-2-7b-chat	100%+0%	N/A	61.8	49.2	18.0	55.0
Llama-3-8b-Instruct	10%+10%	(11, 55)	59.3/63.0	62.2/44.7	98.3/92.0	60.0/73.0
Llama-3-8b-Instruct	5%+10%	(12, 105)	62.8/64.4	60.2/47.3	98.3/92.0	59.0/73.0
Llama-3-8b-Instruct	25%+0%	N/A	60.6/61.5	59.8/44.4	97.3/91.7	51.0/72.0
Llama-3-8b-Instruct	100%+0%	N/A	59.1/61.2	58.0/42.0	95.7/90.9	60.0/73.0
Model	Budget (S + D)	(K,L)	**HotPot**	**Wiki**	**Samsum**	**Trivia**
Llama-2-7b-chat	10%+6%	(11, 120)	35.4	34.5	38.0	83.8
Llama-2-7b-chat	5%+6%	(12, 240)	36.5	34.6	39.6	84.3
Llama-2-7b-chat	25%+0%	N/A	35.3	32.2	38.0	83.6
Llama-2-7b-chat	100%+0%	N/A	37.1	34.7	39.1	83.1
Llama-3-8b-Instruct	10%+10%	(11, 55)	53.3/46.3	40.9/37.1	38.6/37.4	89.2/88.9
Llama-3-8b-Instruct	5%+10%	(12, 105)	53.2/46.0	43.4/36.7	38.4/36.9	89.0/88.8
Llama-3-8b-Instruct	25%+0%	N/A	50.6/46.4	39.9/35.9	37.9/37.7	89.2/88.8
Llama-3-8b-Instruct	100%+0%	N/A	53.5/45.3	39.6/37.0	40.2/39.0	88.4/88.9

6.2 Efficiency

We evaluate the efficiency of MagicPiG on L40 with Intel 6338 CPU as shown in Table 2. We compare the decoding speed with two baselines. (1) For the practical GPU, we slice the requests to the largest mini-batches which can be hosted entirely on GPU. (2) Assuming we have an ideal GPU with infinite VRAM, then we can host a batch of arbitrary size. The practical bandwidth of L40 is estimated as 800GB/s (theoretical maximum 864GB/s) in the comparison.

Conclusions. (1) Through placing the majority of KV cache to CPU RAM, MagicPiG can support a $4 \sim 4.7\times$ larger batch size. (2) Through efficient search and sparse attention, MagicPiG can boost the throughput by $1.6 \sim 2.0\times$ compared to running small batches on GPU and can achieve $80 \sim 97\%$ inference throughput of the ideal GPU with infinite VRAM. In another word, MagicPiG virtualizes CPU RAM as extended VRAM through sparse computation.

7 Next Steps

(1) Conduct experiments on new released long context models (Llama-3.1-8b/70b-128k). Sparsity with super long context models is supposed to be even higher than 4k-8k contexts. (2) CPU related optimizations. Current implementations cannot fully utilize the bandwidth of CPUs. By adding CPU aware optimizations, another $1.5 \sim 2\times$ is hopeful. (3) Conduct experiments on Memory Mosaics.

References

[1] AI@Meta. Llama 3 model card. 2024. URL https://github.com/meta-llama/llama3/blob/main/MODEL_CARD.md.

[2] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.

[3] Leo Gao, Jonathan Tow, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Kyle McDonell, Niklas Muennighoff, Jason Phang, Laria Reynolds, Eric Tang, Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. A framework for few-shot language model evaluation, September 2021. URL https://doi.org/10.5281/zenodo.5371628.

[4] Jiaao He and Jidong Zhai. Fastdecode: High-throughput gpu-efficient llm serving using heterogeneous pipelines. *arXiv preprint arXiv:2403.11421*, 2024.

[5] Daya Khudia, Jianyu Huang, Protonu Basu, Summer Deng, Haixin Liu, Jongsoo Park, and Mikhail Smelyanskiy. Fbgemm: Enabling high-performance low-precision deep learning inference. *arXiv preprint arXiv:2101.05615*, 2021.

[6] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In Jason Flinn, Margo I. Seltzer, Peter Druschel, Antoine Kaufmann, and Jonathan Mace, editors, *Proceedings of the 29th Symposium on Operating Systems Principles, SOSP 2023, Koblenz, Germany, October 23-26, 2023*, pages 611–626. ACM, 2023. doi: 10.1145/3600006.3613165. URL https://doi.org/10.1145/3600006.3613165.

[7] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. Snapkv: Llm knows what you are looking for before generation. *arXiv preprint arXiv:2404.14469*, 2024.

[8] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*, 32, 2019.

[9] Ruoyu Qin, Zheming Li, Weiran He, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. Mooncake: Kimi's kvcache-centric architecture for llm serving. *arXiv preprint arXiv:2407.00079*, 2024.

[10] Prajwal Singhania, Siddharth Singh, Shwai He, Soheil Feizi, and Abhinav Bhatele. Loki: Low-rank keys for efficient sparse attention. *arXiv preprint arXiv:2406.02542*, 2024.

[11] Ryan Spring and Anshumali Shrivastava. A new unbiased and efficient class of lsh-based samplers and estimators for partition function computation in log-linear models. *arXiv preprint arXiv:1703.05160*, 2017.

[12] Jiaming Tang, Yilong Zhao, Kan Zhu, Guangxuan Xiao, Baris Kasikci, and Song Han. Quest: Query-aware sparsity for efficient long-context llm inference. *arXiv preprint arXiv:2406.10774*, 2024.

[13] Gemini Team, Rohan Anil, Sebastian Borgeaud, Yonghui Wu, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M Dai, Anja Hauth, et al. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*, 2023.

[14] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein,

Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurelien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. Llama 2: Open foundation and fine-tuned chat models, 2023.

[15] Chaojun Xiao, Pengle Zhang, Xu Han, Guangxuan Xiao, Yankai Lin, Zhengyan Zhang, Zhiyuan Liu, Song Han, and Maosong Sun. Infllm: Unveiling the intrinsic capacity of llms for understanding extremely long sequences with training-free memory. *arXiv*, 2024.

[16] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. *arXiv preprint arXiv:2309.17453*, 2023.

[17] Zihao Ye, Ruihang Lai, Bo-Ru Lu, Chien-Yu Lin, Size Zheng, Lequn Chen, Tianqi Chen, and Luis Ceze. Cascade inference: Memory bandwidth efficient shared prefix batch decoding, February 2024. URL `https://flashinfer.ai/2024/02/02/cascade-inference.html`.

[18] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for {Transformer-Based} generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 521–538, 2022.

[19] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang "Atlas" Wang, and Beidi Chen. H2o: Heavy-hitter oracle for efficient generative inference of large language models. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, *Advances in Neural Information Processing Systems*, volume 36, pages 34661–34710. Curran Associates, Inc., 2023. URL `https://proceedings.neurips.cc/paper_files/paper/2023/file/6ceefa7b15572587b78ecfcebb2827f8-Paper-Conference.pdf`.